

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение высшего образования  
«Казанский (Приволжский) федеральный университет»  
Институт вычислительной математики и информационных технологий  
Кафедра прикладной математики и искусственного интеллекта

## ОТЧЕТ

Тема: «Многослойный персептрон»

Работу выполнил:  
студент группы 09-322  
Раков Р.О.

Казань 2026

# 1 Введение

Искусственные нейронные сети являются одним из основных инструментов современного машинного обучения. Они позволяют решать задачи классификации, регрессии, распознавания изображений, обработки текста и многие другие задачи.

Целью данного проекта является реализация многослойного персептрона (Multilayer Perceptron, MLP) с нуля на языке Python.

В соответствии с заданием необходимо самостоятельно реализовать основные алгоритмы, лежащие в основе обучения нейронной сети, без использования специализированных библиотек машинного обучения, таких как Keras или PyTorch. Для вычислений использовалась библиотека NumPy, а для анализа и визуализации данных — Pandas, Matplotlib и Seaborn.

В качестве исходного набора данных используется файл data.csv, содержащий характеристики клеток молочной железы, полученные в результате тонкоигольной биопсии. Целевой признак diagnosis принимает два значения:

- В — доброкачественная опухоль (benign);
- М — злокачественная опухоль (malignant).

## 2 Предварительный анализ данных

### 2.1 Описание исходного набора данных

Исходный набор данных представлен в CSV-файле data.csv.

Первый столбец содержит идентификатор наблюдения, второй столбец содержит целевой признак diagnosis. Остальные 30 столбцов содержат числовые характеристики клеток.

Всего в наборе данных содержится:

$$N = 569$$

наблюдений и 30 признаков, используемых для классификации.

В работе идентификатор объекта не используется в качестве признака, поскольку он не содержит информации о характеристиках клетки.

Целевая переменная преобразуется следующим образом:

$$B \rightarrow 0,$$

$$M \rightarrow 1.$$

Таким образом, задача сводится к бинарной классификации.

### 2.2 Проверка пропущенных значений

На первом этапе была выполнена проверка набора данных на наличие пропущенных значений.

В результате анализа было получено:

Количество пропусков = 0.

Следовательно, все 569 объектов содержат значения для всех 30 признаков.

## 2.3 Распределение классов

Распределение целевой переменной оказалось следующим:

- класс В — 357 объектов;
- класс М — 212 объектов.

В процентном выражении:

$$B = 62.74\%,$$

$$M = 37.26\%.$$

Таким образом, классы распределены не идеально равномерно, однако сильного дисбаланса классов нет.

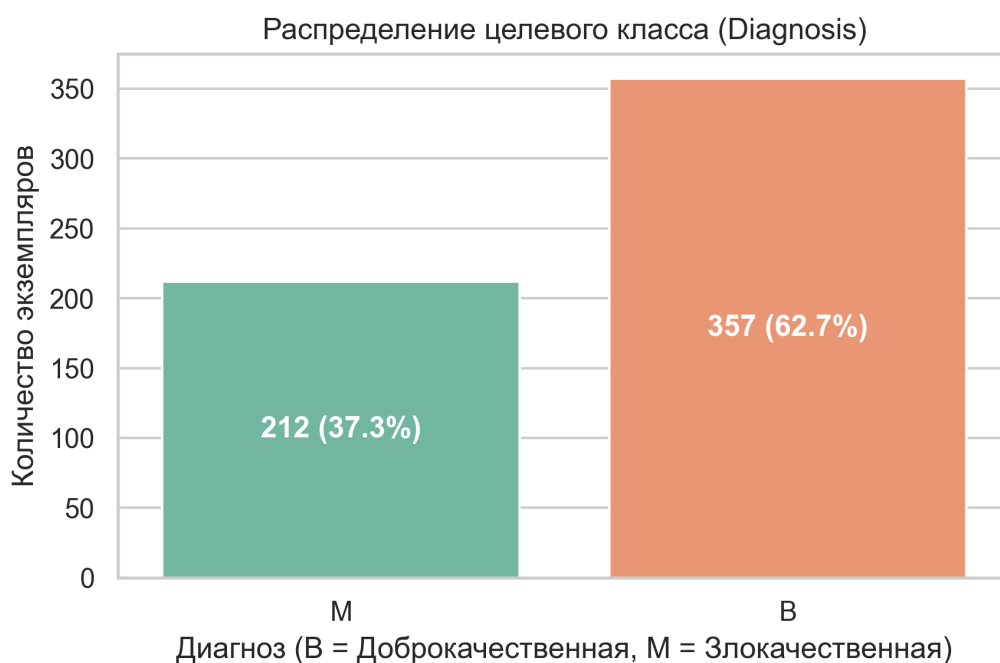


Рисунок 1 — Распределение объектов по классам

На рисунке видно, что доброкачественных объектов больше, чем злокачественных. При этом количество объектов каждого класса достаточно для обучения бинарного классификатора.

## 2.4 Статистический анализ признаков

Для основных признаков были рассчитаны следующие статистические характеристики:

- количество объектов;
- среднее значение;
- стандартное отклонение;
- минимальное значение;
- медианное значение;
- максимальное значение.

В таблице представлены результаты анализа первых десяти признаков.

| Признак                | Кол-во | Среднее  | Стд. откл. | Мин.    | Медиана | Макс.    |
|------------------------|--------|----------|------------|---------|---------|----------|
| radius_mean            | 569    | 14.1273  | 3.5240     | 6.981   | 13.370  | 28.110   |
| texture_mean           | 569    | 19.2896  | 4.3010     | 9.710   | 18.840  | 39.280   |
| perimeter_mean         | 569    | 91.9690  | 24.2990    | 43.790  | 86.240  | 188.500  |
| area_mean              | 569    | 654.8891 | 351.9141   | 143.500 | 551.100 | 2501.000 |
| smoothness_mean        | 569    | 0.0964   | 0.0141     | 0.0526  | 0.0959  | 0.1634   |
| compactness_mean       | 569    | 0.1043   | 0.0528     | 0.0194  | 0.0926  | 0.3454   |
| concavity_mean         | 569    | 0.0888   | 0.0797     | 0.0000  | 0.0615  | 0.4268   |
| concave_points_mean    | 569    | 0.0489   | 0.0388     | 0.0000  | 0.0335  | 0.2012   |
| symmetry_mean          | 569    | 0.1812   | 0.0274     | 0.1060  | 0.1792  | 0.3040   |
| fractal_dimension_mean | 569    | 0.0628   | 0.0071     | 0.0500  | 0.0615  | 0.0974   |

Из таблицы видно, что признаки имеют существенно разные масштабы. Например, area\_mean принимает значения порядка сотен и тысяч, в то время как smoothness\_mean имеет значения порядка сотых.

Поэтому перед обучением нейронной сети необходимо выполнить нормализацию признаков.

## 3 Визуальный анализ данных

### 3.1 Корреляционный анализ

Для изучения взаимосвязей между признаками была построена корреляционная матрица для десяти признаков, относящихся к группе mean.

Коэффициент корреляции Пирсона между двумя признаками  $X$  и  $Y$  вычисляется по формуле:

$$r_{XY} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}.$$

Значение коэффициента находится в диапазоне:

$$-1 \leq r_{XY} \leq 1.$$

Значения, близкие к 1 или -1, свидетельствуют о сильной линейной взаимосвязи между признаками.

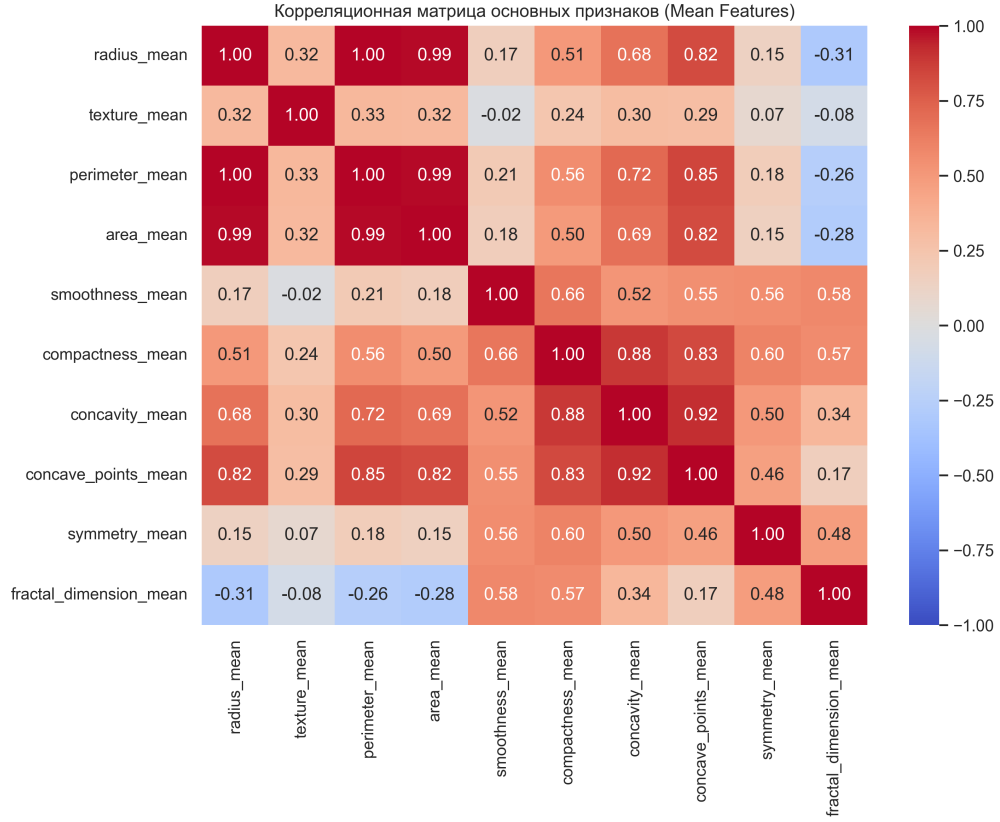


Рисунок 2 — Корреляционная матрица признаков

Из полученной матрицы видно, что некоторые признаки имеют очень высокую положительную корреляцию.

Например:

$$r(\text{radius\_mean}, \text{perimeter\_mean}) \approx 1.00,$$

$$r(\text{radius\_mean}, \text{area\_mean}) \approx 0.99,$$

$$r(\text{concavity\_mean}, \text{concave\_points\_mean}) \approx 0.92.$$

Это означает, что увеличение размера клеточного ядра сопровождается увеличением ряда других геометрических характеристик.

### 3.2 Распределение отдельных признаков

Для визуального сравнения классов были исследованы распределения следующих признаков:

- radius\_mean;

- texture\_mean;
- perimeter\_mean;
- area\_mean.

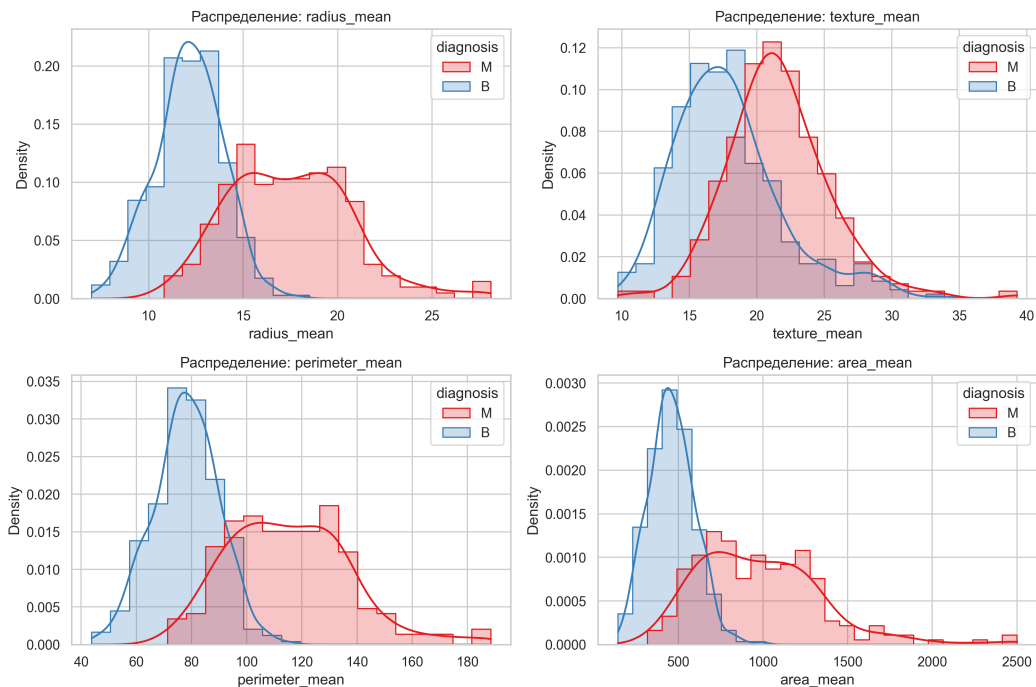


Рисунок 3 — Распределение основных признаков для двух классов

Из графиков видно, что распределения доброкачественных и злокачественных объектов существенно различаются.

В частности, для признаков radius\_mean, perimeter\_mean и area\_mean злокачественные объекты в среднем имеют более высокие значения.

Следовательно, данные признаки содержат информацию, которая может быть полезна для классификации.

## 4 Предварительная обработка данных

### 4.1 Разделение набора данных

Для обучения модели исходный набор был случайным образом перемешан и разделён на три части:

- обучающая выборка — 70%;
- валидационная выборка — 15%;
- тестовая выборка — 15%.

При 569 исходных объектах получилось:

| Выборка    | Количество объектов | Доля |
|------------|---------------------|------|
| Train      | 398                 | 70%  |
| Validation | 85                  | 15%  |
| Test       | 86                  | 15%  |
| Всего      | 569                 | 100% |

Для получения воспроизводимых результатов использовалось фиксированное значение генератора случайных чисел:

$$seed = 42.$$

Использование фиксированного seed позволяет при повторном запуске получить одинаковое перемешивание данных и одинаковое разделение на выборки.

## 4.2 Z-score нормализация

Поскольку исходные признаки имеют различные масштабы, перед подачей данных в нейронную сеть была выполнена стандартизация.

Для каждого признака вычислялись среднее значение и стандартное отклонение только по обучающей выборке:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij},$$

$$\sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2}.$$

После этого каждый признак преобразовывался:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

Важным моментом является то, что  $\mu_j$  и  $\sigma_j$  вычисляются только по обучающей выборке.

Затем эти же параметры применяются к validation и test выборкам.

Такой подход предотвращает утечку информации из проверочных данных в процесс обучения.

## 5 Архитектура многослойного персептрона

В соответствии с условием задания была реализована нейронная сеть, содержащая не менее двух скрытых слоёв.

Используемая архитектура имеет вид:

$$30 \rightarrow 24 \rightarrow 24 \rightarrow 24 \rightarrow 2.$$

Таким образом:

- входной слой — 30 нейронов;
- первый скрытый слой — 24 нейрона;
- второй скрытый слой — 24 нейрона;
- третий скрытый слой — 24 нейрона;
- выходной слой — 2 нейрона.

Три скрытых слоя были выбраны для того, чтобы сеть могла моделировать нелинейные зависимости между входными признаками.

Архитектура задаётся программно и может быть изменена через аргументы командной строки.

## 6 Прямое распространение

Прямое распространение (Forward Propagation) используется для получения предсказания нейронной сети.

Для каждого слоя сначала вычисляется линейная комбинация:

$$z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)},$$

где:

- $a^{(l-1)}$  — выход предыдущего слоя;
- $W^{(l)}$  — матрица весов;
- $b^{(l)}$  — вектор смещений;
- $z^{(l)}$  — значение до применения функции активации.

После этого применяется функция активации:

$$a^{(l)} = f(z^{(l)}).$$

Для скрытых слоёв используется сигмоидальная функция:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Производная сигмоиды имеет удобный вид:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

В программе производная вычисляется через уже полученное значение активации:

$$\sigma'(z) = a(1 - a).$$

## 7 Функция Softmax

Согласно заданию, выходной слой должен использовать функцию softmax, позволяющую получить вероятностное распределение по классам.

Для  $K$  классов функция softmax имеет вид:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}.$$

Для двух классов получаются два значения:

$$p_0 + p_1 = 1.$$

Поэтому выход нейронной сети можно интерпретировать как вероятности принадлежности объекта к каждому классу.

В программе для численной устойчивости перед вычислением экспоненты из всех значений вычитается максимальное:

$$\text{softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_j e^{z_j - \max(z)}}.$$

Это предотвращает переполнение при больших значениях  $z$ .

## 8 Функция потерь

Для оценки качества классификации используется бинарная кросс-энтропия.

В реализации выходной слой содержит два значения, поэтому целевая переменная представляется в one-hot формате.

Например:

$$B \rightarrow [1, 0],$$

$$M \rightarrow [0, 1].$$

Функция потерь имеет вид:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^2 y_{ik} \ln(\hat{y}_{ik}).$$

Для предотвращения вычисления  $\ln(0)$  значения вероятностей ограничиваются небольшим диапазоном:

$$\varepsilon \leq \hat{y} \leq 1 - \varepsilon,$$

где

$$\varepsilon = 10^{-15}.$$

Чем меньше значение функции потерь, тем ближе предсказанные вероятности к правильным значениям классов.

## 9 Обратное распространение ошибки

Для обучения нейронной сети используется алгоритм обратного распространения ошибки (Backpropagation).

Его задача заключается в вычислении градиентов функции потерь по отношению к весам и смещениям всех слоёв.

Для выходного слоя при использовании softmax совместно с кросс-энтропией градиент имеет простой вид:

$$\delta^{(L)} = \frac{\hat{y} - y}{m},$$

где:

- $\hat{y}$  — предсказание сети;
- $y$  — правильный ответ;
- $m$  — размер текущего мини-батча.

Градиент по матрице весов вычисляется:

$$\frac{\partial L}{\partial W^{(l)}} = (a^{(l-1)})^T \delta^{(l)}.$$

Градиент по смещению:

$$\frac{\partial L}{\partial b^{(l)}} = \sum_i \delta_i^{(l)}.$$

Для скрытого слоя ошибка передаётся назад:

$$\delta^{(l)} = (\delta^{(l+1)}(W^{(l+1)})^T) \odot \sigma'(z^{(l)}),$$

где  $\odot$  обозначает поэлементное умножение.

## 10 Градиентный спуск

После вычисления градиентов веса и смещения обновляются методом градиентного спуска.

Для весов:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}.$$

Для смещений:

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}.$$

Здесь  $\eta$  — скорость обучения.

В проведённом эксперименте использовалось:

$$\eta = 0.0314.$$

Обучение выполнялось мини-батчами размером:

$$batch\_size = 8.$$

Количество эпох:

$$epochs = 84.$$

Таким образом, на каждой эпохе обучающая выборка перемешивается, разбивается на мини-батчи и для каждого мини-батча выполняются:

1. прямое распространение;
2. вычисление ошибки;
3. обратное распространение;
4. обновление весов и смещений.

## 11 Программная реализация

В соответствии с заданием программа была разделена на несколько частей:

1. `split.py` — подготовка и разделение данных;
2. `train.py` — создание и обучение нейронной сети;
3. `predict.py` — загрузка обученной модели и оценка на тестовой выборке;
4. `analysis.py` — дополнительная программа анализа данных.

### 11.1 Программа подготовки данных

Файл `split.py` выполняет следующие операции:

1. загружает исходный CSV-файл;
2. отделяет целевой признак;
3. удаляет идентификатор из набора признаков;
4. преобразует значения М/В в 1/0;

5. перемешивает данные;
6. разделяет данные на Train, Validation и Test;
7. вычисляет параметры стандартизации по Train;
8. нормализует все три выборки;
9. сохраняет подготовленные данные в dataset\_split.npz.

Фрагмент преобразования целевой переменной:

```
1 y = (df.iloc[:, 1].values == 'M').astype(int)
2 X = df.iloc[:, 2:].values.astype(float)
```

Разделение данных:

```
1 train_end = int(0.70 * n_samples)
2 valid_end = int(0.85 * n_samples)
3
4 train_idx = indices[:train_end]
5 valid_idx = indices[train_end:valid_end]
6 test_idx = indices[valid_end:]
```

Нормализация:

```
1 mean = np.mean(X_train_raw, axis=0)
2 std = np.std(X_train_raw, axis=0)
3
4 X_train = (X_train_raw - mean) / std
5 X_valid = (X_valid_raw - mean) / std
6 X_test = (X_test_raw - mean) / std
```

## 11.2 Класс DenseLayer

Каждый полносвязный слой представлен классом DenseLayer.

Слой содержит:

- матрицу весов  $W$ ;
- вектор смещений  $b$ ;
- название функции активации.

Для инициализации весов используется равномерное распределение с границами:

$$\pm\sqrt{\frac{6}{n}},$$

где  $n$  — количество входов слоя.

В программе:

```
1 limit = np.sqrt(6.0 / input_dim)
2
3 self.W = np.random.uniform(
```

```

4     -limit ,
5     limit ,
6     (input_dim , output_dim)
7 )
8
9 self.b = np.zeros((1, output_dim))

```

### 11.3 Класс MLP

Класс MLP создаёт набор слоёв на основании переданной архитектуры.

Для использованной конфигурации:

```

1 architecture = [30, 24, 24, 24, 2]

```

создаются три скрытых слоя с функцией sigmoid и выходной слой с softmax.

Таким образом, архитектура может изменяться без переписывания алгоритма нейронной сети.

## 12 Результаты обучения

Обучение проводилось с параметрами:

| Параметр                         | Значение              |
|----------------------------------|-----------------------|
| Архитектура                      | 30 – 24 – 24 – 24 – 2 |
| Количество эпох                  | 84                    |
| Размер мини-батча                | 8                     |
| Learning rate                    | 0.0314                |
| Seed                             | 42                    |
| Функция активации скрытых слоёв  | Sigmoid               |
| Функция активации выходного слоя | Softmax               |

В процессе обучения на каждой эпохе рассчитывались:

- training loss;
- validation loss;
- training accuracy;
- validation accuracy.

Первые эпохи обучения характеризуются быстрым улучшением качества модели.

Например, после первой эпохи:

$$Accuracy_{train} = 64.82\%,$$

$$Accuracy_{valid} = 51.76\%.$$

К 10-й эпохе:

$$Accuracy_{train} = 89.20\%,$$

$$Accuracy_{valid} = 88.24\%.$$

К 25-й эпохе:

$$Accuracy_{train} = 98.49\%,$$

$$Accuracy_{valid} = 95.29\%.$$

К концу обучения:

$$Accuracy_{train} = 99.25\%,$$

$$Accuracy_{valid} = 96.47\%.$$

При этом значение функции потерь на обучающей выборке уменьшилось с примерно 0.65 до:

$$Loss_{train} = 0.0436.$$

На валидационной выборке в последнюю эпоху:

$$Loss_{valid} = 0.1591.$$

## 12.1 Кривая функции потерь и Кривая точности

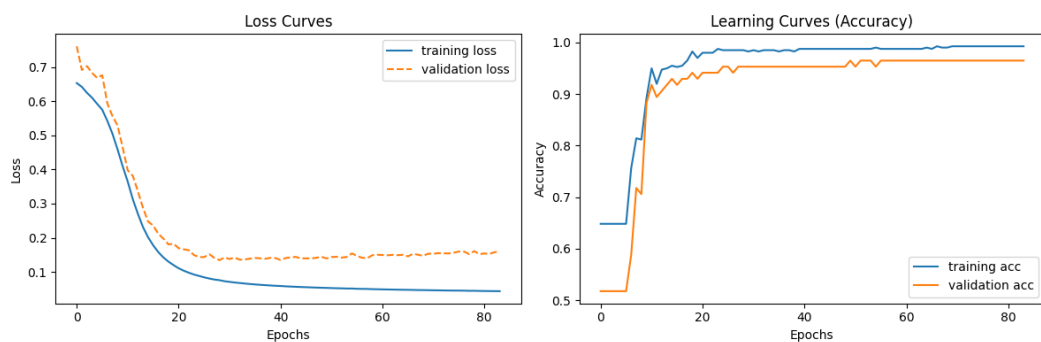


Рисунок 4 — Кривые функции потерь в процессе обучения и Изменение точности модели в процессе обучения

На графике видно, что функция потерь на обучающей выборке практически монотонно уменьшается на протяжении всего обучения.

Validation loss быстро уменьшается в первые эпохи, после чего начинает колебаться около значения  $0.14 - 0.16$ .

На графике видно быстрое увеличение точности в первые 20–25 эпох.

После этого validation accuracy стабилизируется около 95–96%, в то время как training accuracy продолжает постепенно увеличиваться.

К концу обучения разница между обучающей и валидационной точностью составляет:

$$99.25\% - 96.47\% = 2.78\%.$$

## 13 Сохранение обученной модели

После завершения обучения модель сохраняется в файл:

saved\_model.npy

В файл сохраняются:

- архитектура нейронной сети;
- названия функций активации;
- матрицы весов;
- векторы смещений.

Структура сохраняемых данных имеет вид:

```
1 model_data = {  
2     'architecture': architecture,  
3     'activations': [l.activation_name  
4                     for l in model.layers],  
5     'W': [l.W for l in model.layers],  
6     'b': [l.b for l in model.layers]  
7 }
```

Таким образом, после завершения обучения нет необходимости обучать сеть заново для выполнения прогнозирования.

## 14 Программа прогнозирования

Программа predict.py загружает:

1. тестовую выборку из файла dataset\_split.npz;
2. архитектуру сети;
3. веса;
4. смещения;

5. функции активации.

После этого выполняется прямое распространение для тестовой выборки.

Полученные вероятности преобразуются в предсказанный класс с помощью:

$$\hat{y} = \arg \max_k p_k.$$

Таким образом, выбирается класс, вероятность которого является максимальной.

## 15 Оценка качества на тестовой выборке

После завершения обучения была выполнена независимая оценка модели на тестовой выборке.

Количество тестовых объектов:

$$N_{test} = 86.$$

Полученные результаты:

$$Loss_{test} = 0.0760,$$

$$Accuracy_{test} = 97.67\%.$$

Результаты представлены в таблице.

| Метрика                        | Значение |
|--------------------------------|----------|
| Test Binary Cross-Entropy Loss | 0.0760   |
| Test Accuracy                  | 97.67%   |

Так как тестовая выборка содержит 86 объектов, точность 97.67% соответствует примерно 84 правильно классифицированным объектам.

$$86 \cdot 0.9767 \approx 84.$$

Следовательно, модель показала высокое качество классификации на тестовой выборке.

## 16 Сравнение результатов обучения и тестирования

Полученные результаты можно представить следующим образом:

| Выборка    | Accuracy | Loss   |
|------------|----------|--------|
| Training   | 99.25%   | 0.0436 |
| Validation | 96.47%   | 0.1591 |
| Test       | 97.67%   | 0.0760 |

Полученный результат показывает, что модель способна хорошо обобщать закономерности, обнаруженные на обучающей выборке.

## 17 Заключение

В рамках проекта был реализован многослойный персептрон для решения задачи бинарной классификации характеристик клеток молочной железы.

На первом этапе был выполнен предварительный анализ исходного набора данных. Было установлено, что набор содержит 569 объектов, 30 числовых признаков и целевой признак с двумя классами: В и М. Пропущенные значения отсутствуют.

Анализ распределений показал, что классы имеют различные распределения по основным признакам. В частности, признаки, характеризующие размер клеточного ядра, имеют заметные различия между доброкачественными и злокачественными объектами.

Перед обучением данные были разделены на обучающую, валидационную и тестовую выборки в пропорции 70/15/15. Для устранения различий в масштабах признаков была выполнена Z-score нормализация на основе статистик обучающей выборки.

Была реализована нейронная сеть архитектуры

$$30 - 24 - 24 - 24 - 2,$$

с тремя скрытыми слоями. В скрытых слоях использовалась функция sigmoid, а на выходном слое — softmax.

Основные алгоритмы нейронной сети были реализованы самостоятельно с использованием NumPy. Были реализованы прямое распространение, обратное распространение ошибки и градиентный спуск с мини-батчами.

В процессе обучения модель достигла следующих результатов:

$$Accuracy_{train} = 99.25\%,$$

$$Accuracy_{valid} = 96.47\%.$$

После загрузки сохранённых весов была выполнена независимая оценка на тестовой выборке:

$$Loss_{test} = 0.0760,$$

$$Accuracy_{test} = 97.67\%.$$

Полученный результат показывает, что реализованный многослойный персептрон успешно решает поставленную задачу бинарной классификации.

В ходе работы были изучены и практически реализованы основные этапы обучения нейронной сети: подготовка данных, нормализация, прямое распространение, вычисление функции потерь, обрат-

ное распространение ошибки, вычисление градиентов и обновление параметров методом градиентного спуска.

Таким образом, поставленная цель проекта была достигнута.